Introdução ao Hibernate

Conteudista: Prof. Me. Alexander Gobbato Paulino Albuquerque
Revisão Textual: Prof.ª Esp. Lorena Garcia Aragão de Souza

Objetivos da Unidade:

- Introduzir o *Hibernate*;
- Explicar como funciona o mapeamento e as operações usando o *Hibernate*.

Material Teórico

Material Complementar

Referências

Material Teórico

Entendendo o *Hibernate*

O *Hibernate* é um *framework* para o mapeamento objeto-relacional, escrito na linguagem *Java*, mas também é disponível em .Net (ADO.NET) com o nome *NHibernate*.

O *NHibernate Driver* é um *driver* específico para acesso a determinados bancos de dados. Ele utiliza as interfaces ou conjunto de classes que expõem serviços de acesso a dados. Interfaces como *IDbConnection*, *IDbCommand*, *IDbDataReader* e *IDataParameter*

Esse *framework* facilita o mapeamento dos atributos entre uma base tradicional de dados relacionais e o modelo objeto de uma aplicação, mediante o uso de arquivos (XML) ou anotações *Java*.

Hibernate é um *software* livre de código aberto, distribuído com a licença LGPL.

Características

O objetivo do *Hibernate* é diminuir a complexidade entre os programas *Java*, baseado no modelo orientado a objeto, que precisa trabalhar com um banco de dados do modelo relacional (presente na maioria dos SGBDs). Em especial, no desenvolvimento de consultas e atualizações dos dados.

Sua principal característica é a transformação das classes em *Java* para tabelas de dados (e dos tipos de dados *Java* para os da SQL). O *Hibernate* gera as chamadas SQL e libera o desenvolvedor

do trabalho manual da conversão dos dados resultantes, mantendo o programa portável para quaisquer bancos de dados SQL, porém causando um pequeno aumento no tempo de execução.

HQL (Hibernate Query Language)

A *HQL* (*Hibernate Query Language*) é um dialeto SQL para o *Hibernate*. Ela é uma poderosa linguagem de consulta, que se parece muito com a SQL, mas a HQL é totalmente orientada a objeto, incluindo os paradigmas de herança, polimorfismo e encapsulamento.

No *Hibernate*, você pode escolher usar tanto a SQL quanto a HQL. Escolhendo a HQL, você poderá executar os pedidos SQL sobre as classes de persistência do *Java* em vez de tabelas no banco de dados.

Utilizando o HQL temos a vantagem de portabilidade de banco, ou seja, suponha que estamos utilizando um banco de dados A. Ao trocarmos para um banco B, o HQL automaticamente cria comandos referentes a cada banco de dados. Isso facilita, pois no SQL teríamos que rastrear e alterar vários códigos no sistema.

Hibernate

O *Hibernate* é um *framework* para persistir ou salvar objetos *Java* em um banco de dados. É um *framework* muito popular, usado por muitos projetos *Java* corporativos e você pode baixá-lo gratuitamente do modo de hibernação.org.

Basicamente, em nível muito alto, você terá seu aplicativo *Java*, ele fará uso dessa estrutura do *Hibernate* e poderá usá-lo para salvar e recuperar dados do banco de dados.

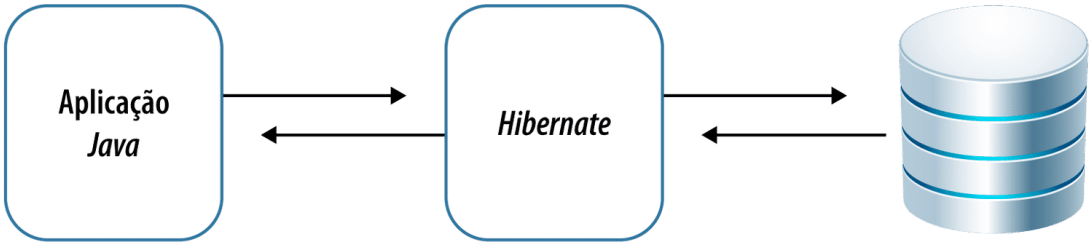


Figura 1 – Transformando comando em *Java* e Instruções de banco

Quais os Benefícios do *Hibernate*?

O *Hibernate* fornece algo chamado mapeamento de objeto para relacional ou você ouvirá a palavra de ordem ou um chamado ORM.

Como um desenvolvedor, você precisa dizer ao *Hibernate* como sua classe *Java* ou como ele mapeia os dados no banco de dados.

Na verdade, você mapeará sua classe *Java* para uma determinada tabela de banco de dados.

ORM (Object to Relational Mapping)

Veja, temos nossa classe *Java Student*, ela possui quatro campos: id, *firstName*, *lastName* e *e-mail*.

Temos a estrutura do *Hibernate* no meio e na direita temos a tabela real do banco de dados.

Na tabela temos o id, que é a chave primária, first_name e last_name. Observe seu primeiro nome de sublinhado, último nome de sublinhado e um campo para um endereço de e-mail.

O que o *Hibernate* irá fazer é mapear a classe *student* para essa tabela.

Será configurado o mapeamento um-para-um entre os campos e as colunas reais no banco de dados.

Podemos configurar esse mapeamento por meio de um arquivo de configuração ou usando XML ou você pode configurá-lo usando anotações *Java*.

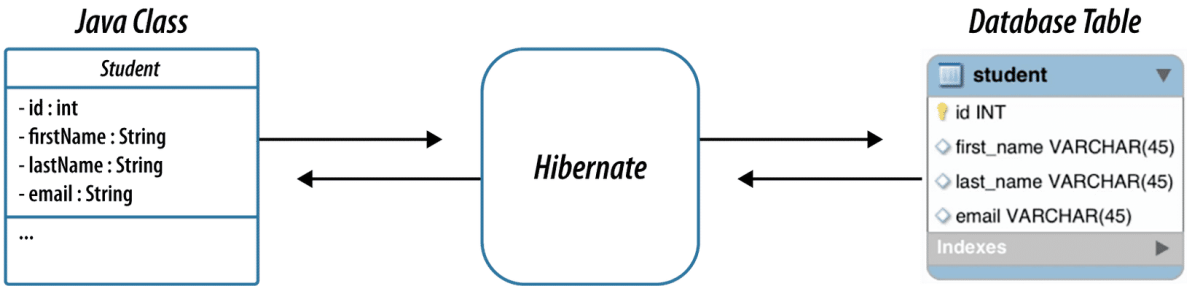


Figura 2 – Relação *Java* com Banco de Dados

Hibernate e o JDBC

O *Hibernate* realmente usa o JDBC para todas as comunicações do banco de dados, então, na verdade, o *Hibernate* é apenas outra camada de extração em cima do JDBC.

Logo, quando seu aplicativo usa a estrutura do *Hibernate*, seu aplicativo armazenará e recuperará objetos usando a API do *Hibernate*.

Em segundo plano, o *Hibernate* faz todo o trabalho JDBC de baixo nível e envia as consultas SQL e assim por diante.

Então, o *Hibernate* faz muito do trabalho de baixo nível, mas em segundo plano, tudo passa pela API JDBC padrão.

Quando configuramos o *Hibernate* para conversar com o banco de dados, na verdade vamos configurar o *Hibernate* para fazer uso de um *driver* JDBC.

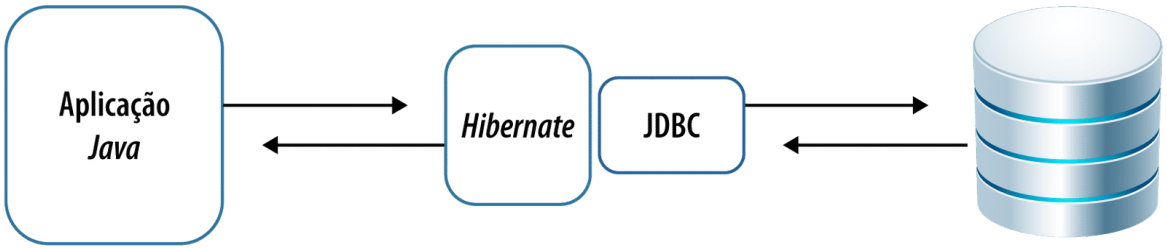


Figura 3 – Explicação prática dos elementos *Java* com a chamada ao banco de dados

---

Hibernate

- *Hibernate* 5.2.;
- *Hibernate* 5.2 requer *Java* 8.

Configurando o *Hibernate*

- Banco de Dados *MySql*;
- *Hibernate Jar*;
- JDBC *Driver*.

Configurando o *Hibernate* com Eclipse

- Criar um projeto no Eclipse (mudar perspectiva para *java*);
- Copiar os arquivos do *Hibernate* (.jar);

- Copiar o *driver* do JDBC.

Hibernate.org

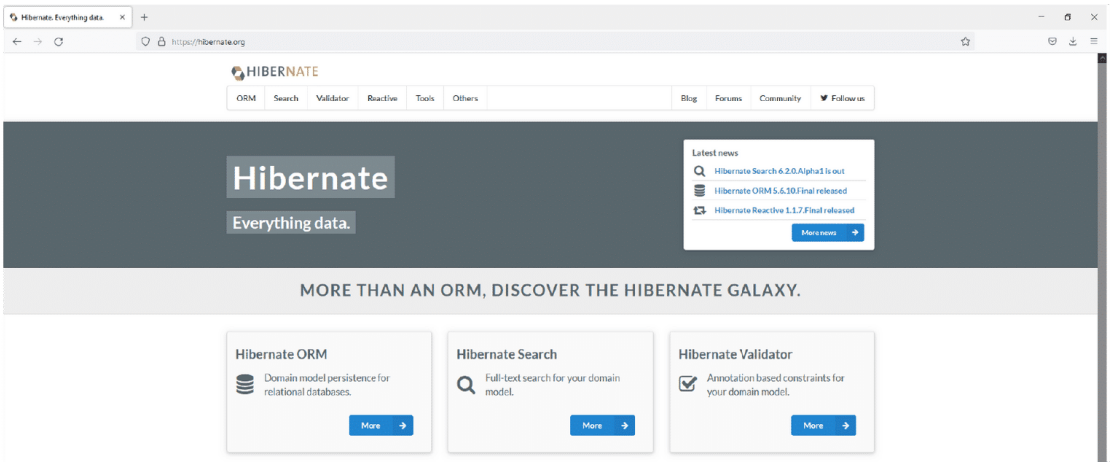


Figura 4 – Acessando o *framework*
Fonte: Reprodução

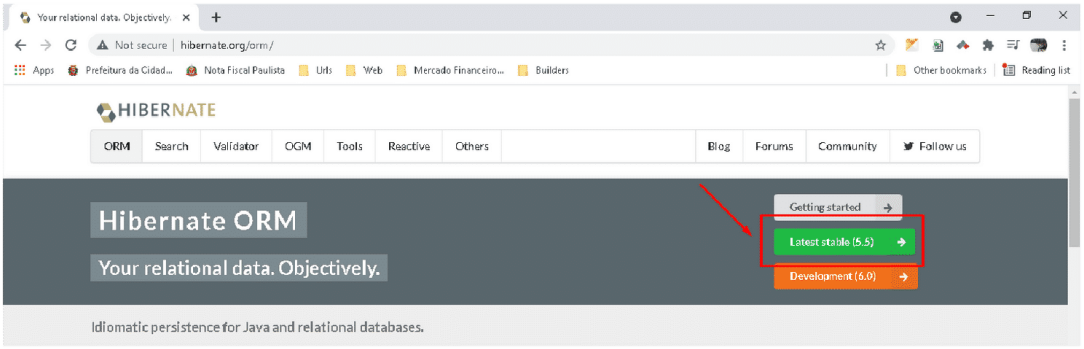


Figura 5 – Versão estável
Fonte: Reprodução

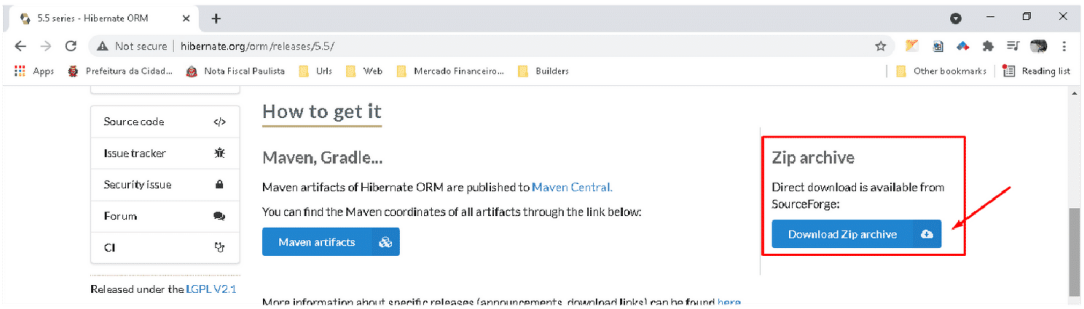


Figura 6 – *Download* dos arquivos
Fonte: Reprodução

Arquivos Necessários

Após o *download* e a descompactação, os arquivos necessários são:

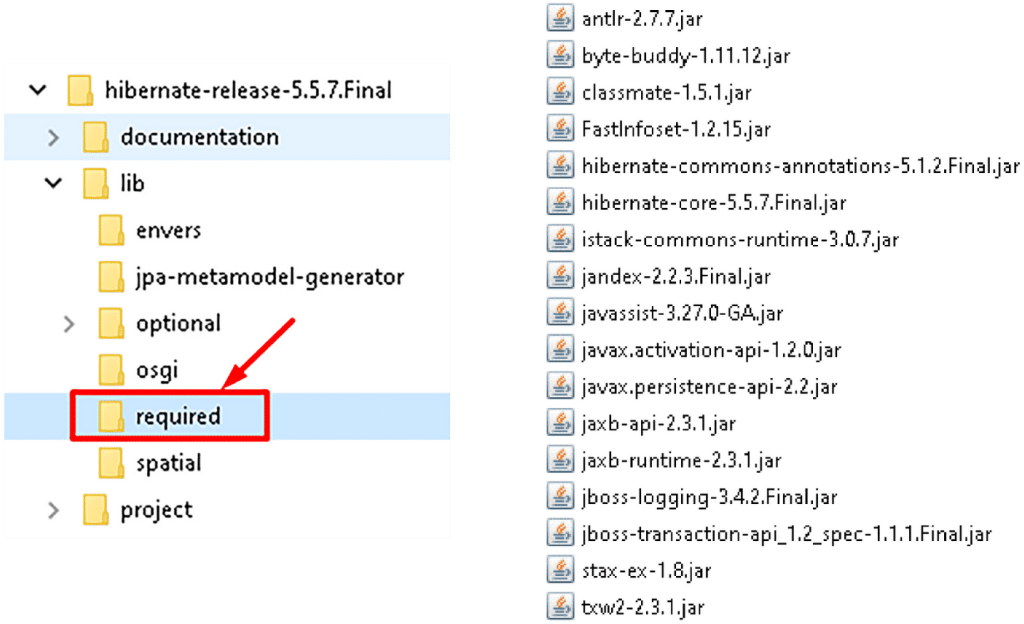


Figura 7 – Ajustando os arquivos no projeto
Fonte: Reprodução

Configurando Eclipse

Mudar a perspectiva do projeto:

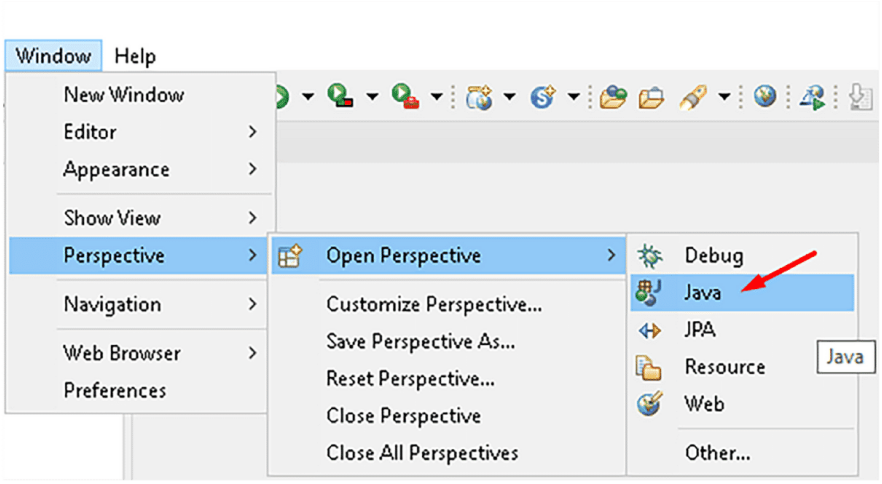


Figura 8 – Alterando perspectiva
Fonte: Reprodução

Criar um projeto *Java*:

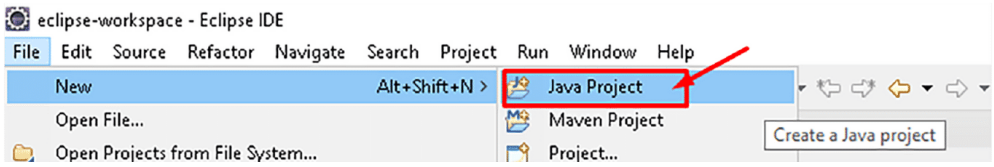


Figura 9 – Criando o projeto
Fonte: Reprodução

Criar uma pasta com o nome lib:

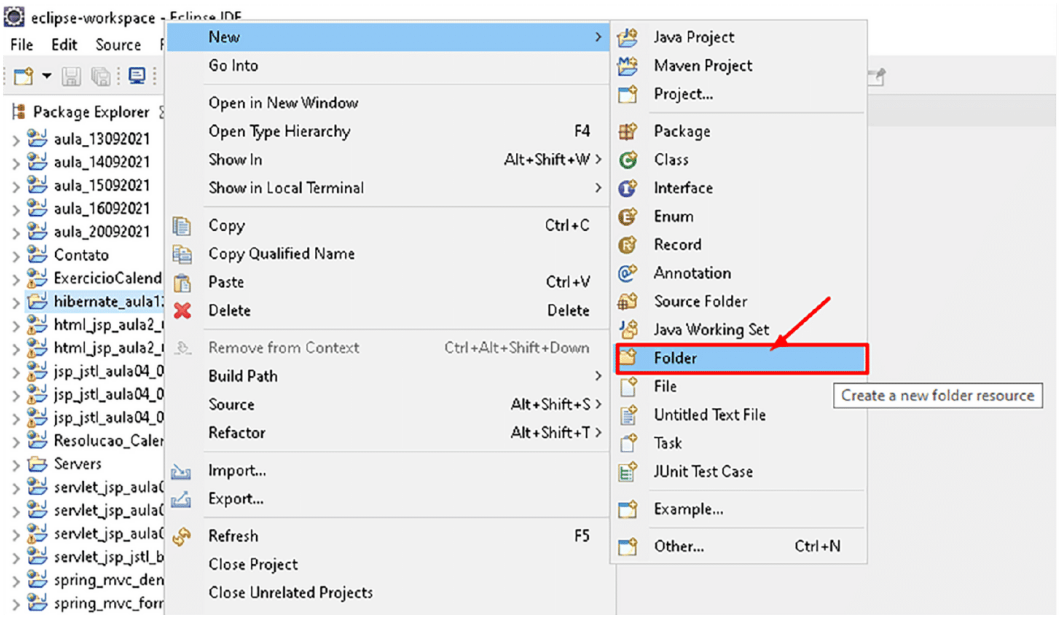


Figura 10 – Criando pasta para repositório
Fonte: Reprodução

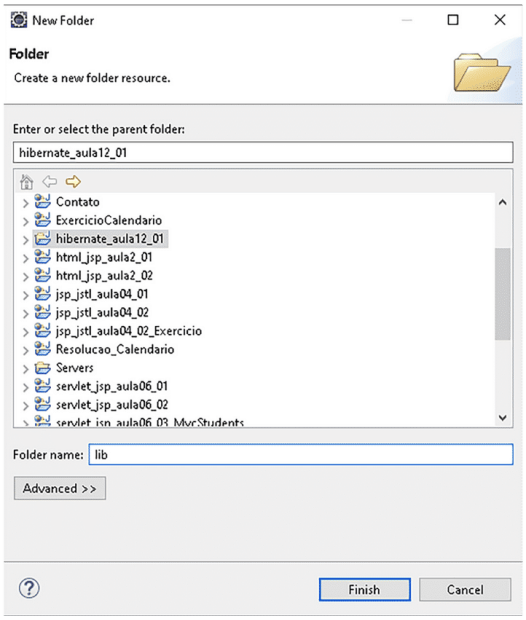


Figura 11 – Selecionando pasta destino
Fonte: Reprodução

Configurando Eclipse

Na pasta lib criada, copiar os arquivos do *hibernate* e o *driver* JDBC do *mysql*:

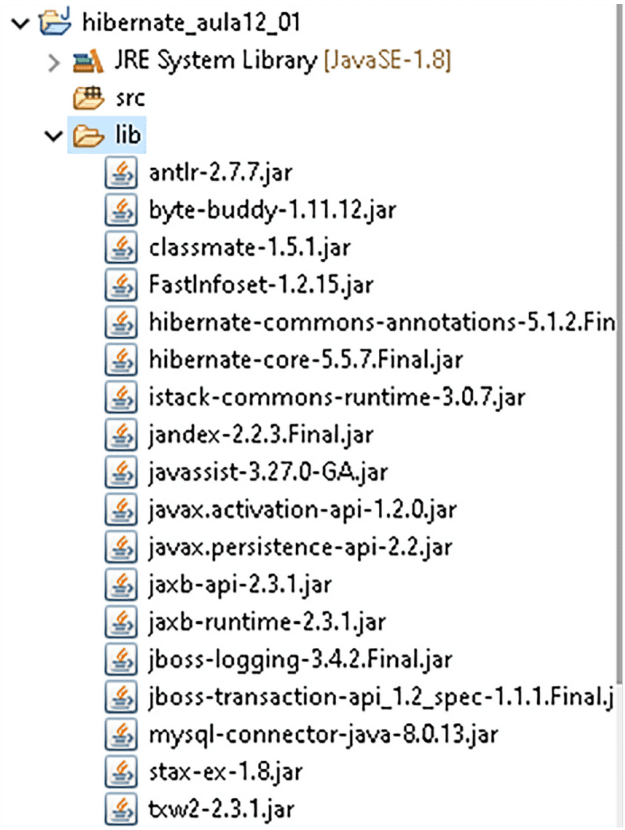


Figura 12 – Configurando a pasta de biblioteca do projeto
Fonte: Reprodução

Configurar o *Java Build Path*. Clicar com o botão direito no projeto e selecionar *Properties*:

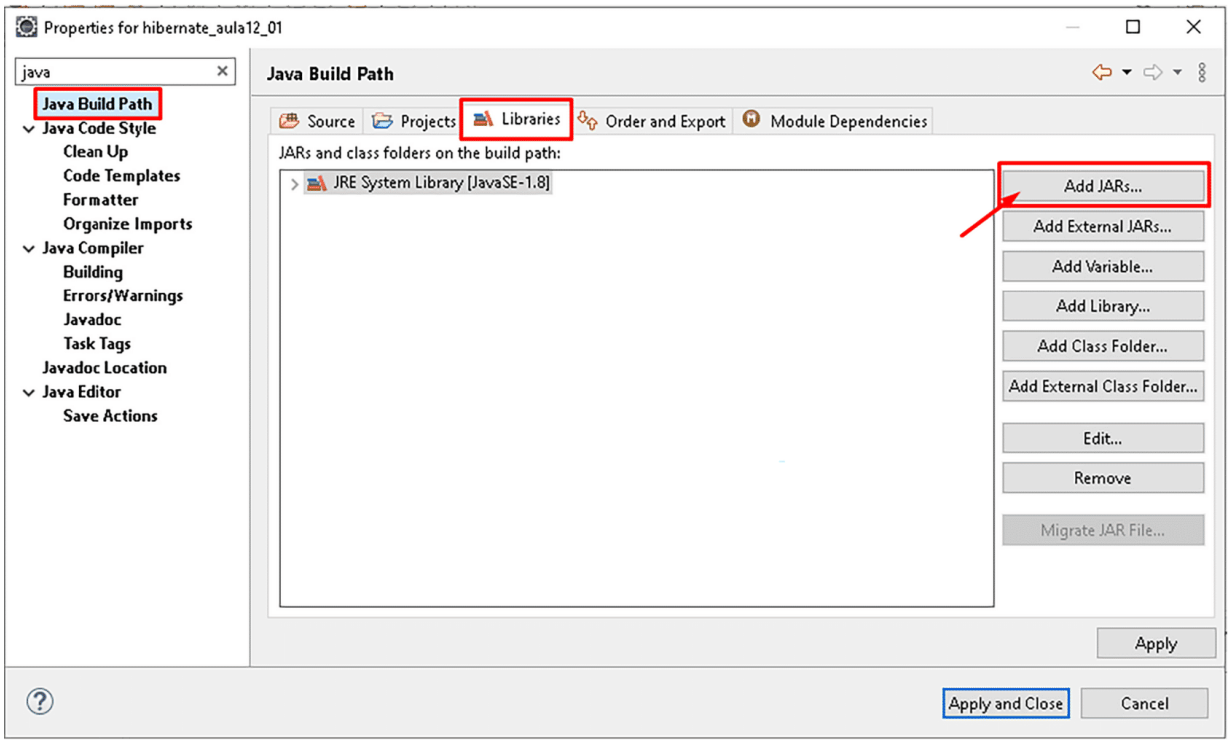


Figura 13 – Adicionando os arquivos do *framework*
Fonte: Reprodução

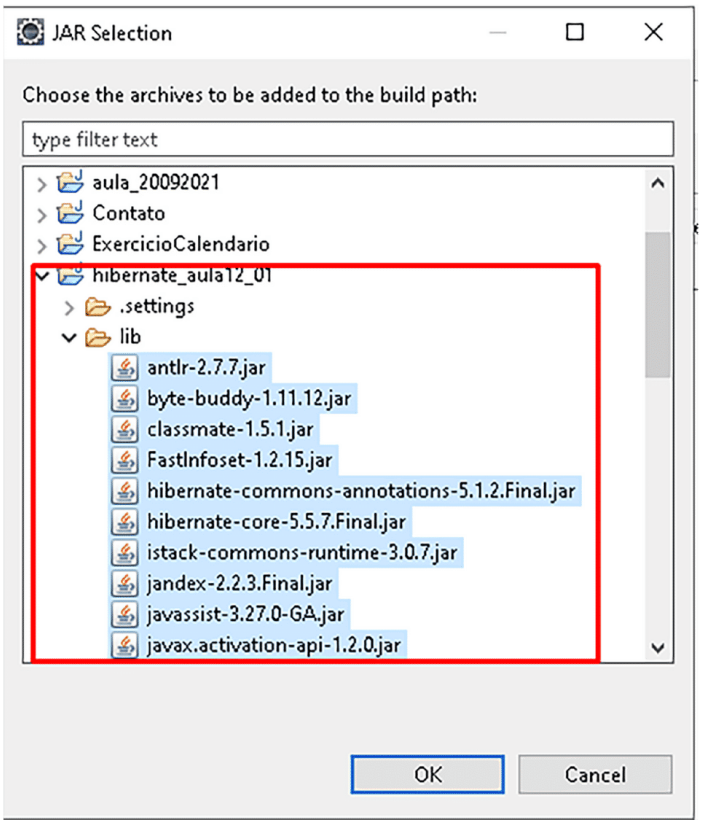


Figura 14 – Selecionando os arquivos
Fonte: Reprodução

Testando a Conexão com o Banco

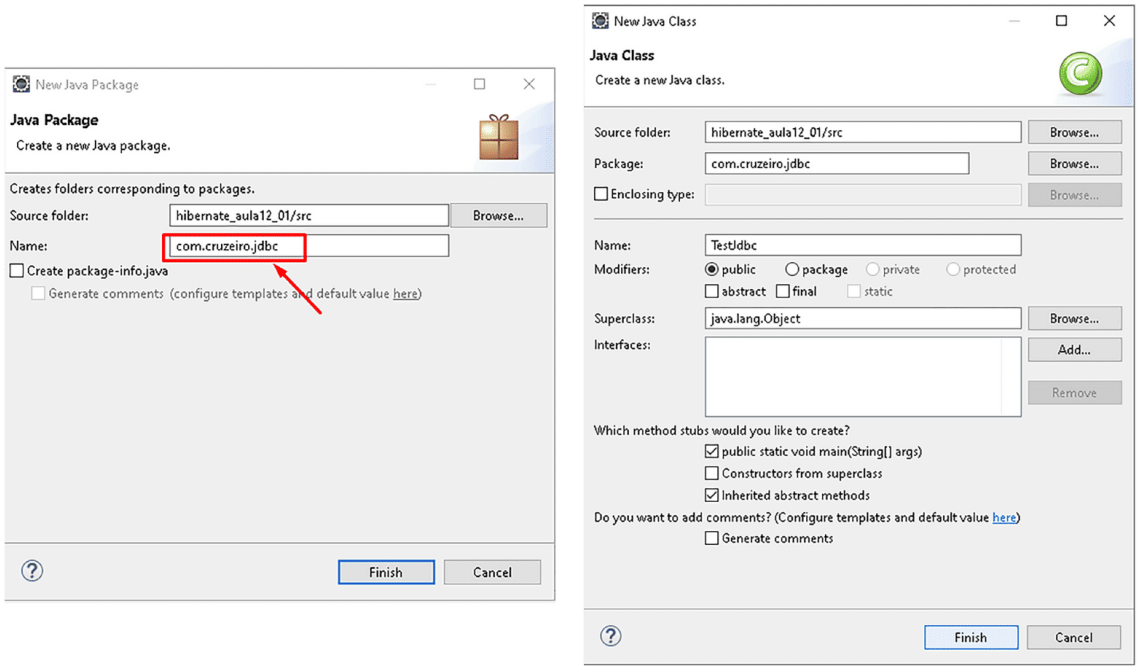


Figura 15 – Configurando pacote e criando teste
Fonte: Reprodução

Criando o código no eclipse:

```
package com.cruzeiro.jdbc;
import java.sql.Connection;
import java.sql.DriverManager;
public class TestJdbc {
public static void main(String[] args) {
String jdbcUrl = "jdbc:mysql://localhost:3306/hb_student_tracker?useSSL=fal
String user = "hbstudent";
String pass = "hbstudent";
try {
        System.out.println("Connecting to database: " + jdbcUrl);
        Connection myConn = DriverManager.getConnection(jdbcUrl, user, pass
        System.out.println("Connection successful!!!");
}
catch (Exception exc) {
        exc.printStackTrace();
```

```
    }
    }
    }
```

Vamos entender o código que é o teste de conexão.

É necessário a criação de uma variável, na qual passamos o endereço, o usuário e a senha do banco de relacional, no nosso caso, o *mysql*.

Processo de Desenvolvimento do *Hibernate*

Lista de etapas:

1. Adicionar o arquivo de configuração do *hibernate*;
2. Criar o *notation* do *Java*;
3. Desenvolver os códigos para manipulação no banco.

Etapa 1 – Criando o Arquivo de Configuração do *Hibernate*

A seguir, estamos criando um arquivo de configuração xml que é muito parecido com o exemplo que utilizamos em *Java* anteriormente.

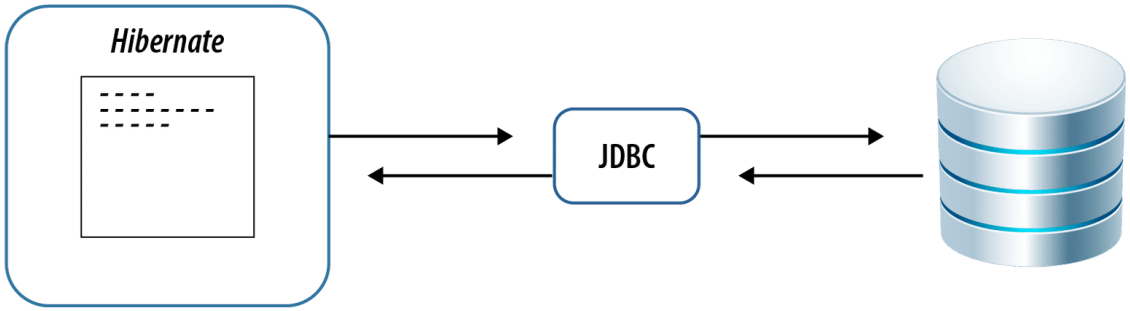


Figura 16 – Configurando o xml
Fonte: Reprodução

No *hibernate-configuration* precisamos informar o *driver*, o caminho, o usuário e a senha do banco. Nesse arquivo de configuração, usaremos o *dialect* para informar qual o banco de dados iremos utilizar, por isso o *org.hibernate.dialect.MySqlDialect*.

```
<!DOCTYPE hibernate-configuration PUBLIC
        "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
        "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
<hibernate-configuration>
    <session-factory>
        <!-- JDBC Database connection settings -->
        <property name="connection.driver_class">com.mysql.cj.jdbc.Driver</
        <propertyname="connection.url">
        jdbc:mysql://localhost:3306/hb_student_tracker?useSSL=false&amp;ser
        <property name="connection.username">hbstudent</property>
        <property name="connection.password">hbstudent</property>
        <!-- JDBC connection pool settings ... using built-in test pool -->
        <property name="connection.pool_size">1</property>
        <!-- Select our SQL dialect -->
        <property name="dialect">org.hibernate.dialect.MySQLDialect</proper
        <!-- Echo the SQL to stdout -->
        <property name="show_sql">true</property>
        <!-- Set the current session context -->
        <property name="current_session_context_class">thread</property>
    </session-factory>
</hibernate-configuration>
```

Criar o *Notation* do *Java*

À medida que a terminologia *hibernate* é utilizada, entendemos que é apenas um nome sofisticado para uma classe *Java* que é mapeada para uma tabela de banco de dados, e é realmente apenas uma classe *Java* com campos e métodos *getters* e *setters*, e você simplesmente adiciona anotações para ajudar a mapeá-la para uma tabela de banco de dados.

Pense nele como uma classe *Java* antiga comum ou um pojo, com anotações especiais.

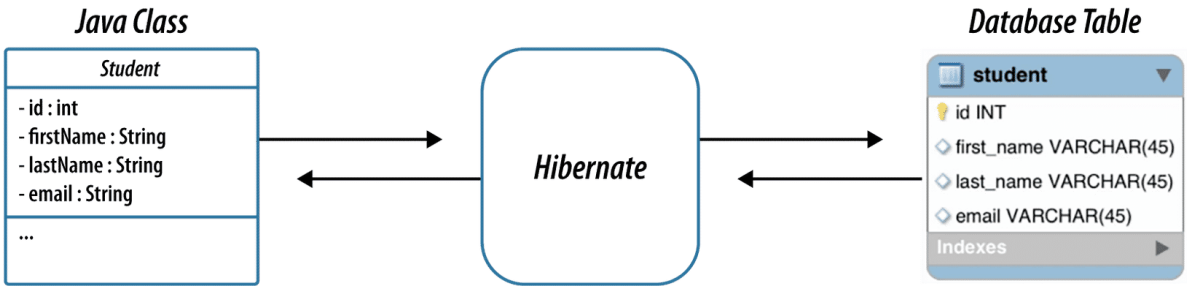


Figura 17 – Criando *Notation*
Fonte: Reprodução

Existem duas formas de mapear as entidades:

1. Arquivo de configuração XML (antiga);
2. *Java Annotations* (moderna).

Java Annotions

1. Mapear a classe para a tabela no banco de dados;

2 Mapear os atributos da classe com as colunas da tabela no banco de dados.

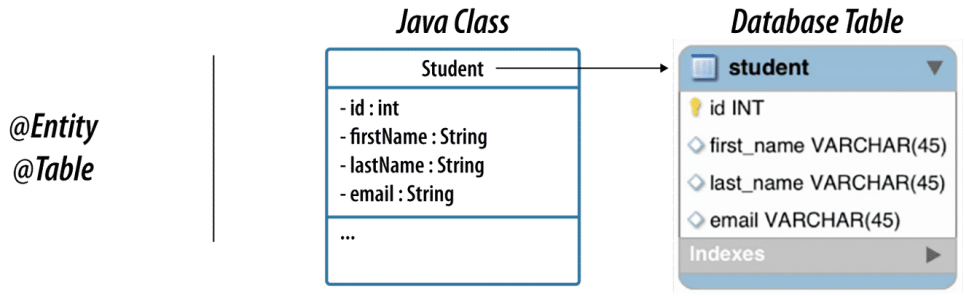


Figura 18 – Mapeamento classe e tabela
Fonte: Reprodução

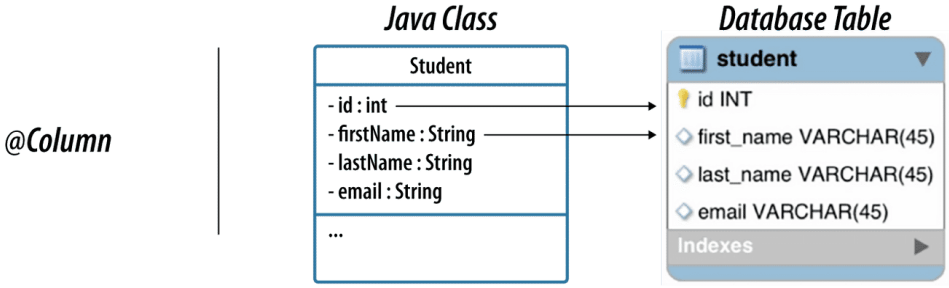


Figura 19 – Mapeando atributos
Fonte: Reprodução

Desenvolver os Códigos para Manipulação no Banco

Criar um pacote:

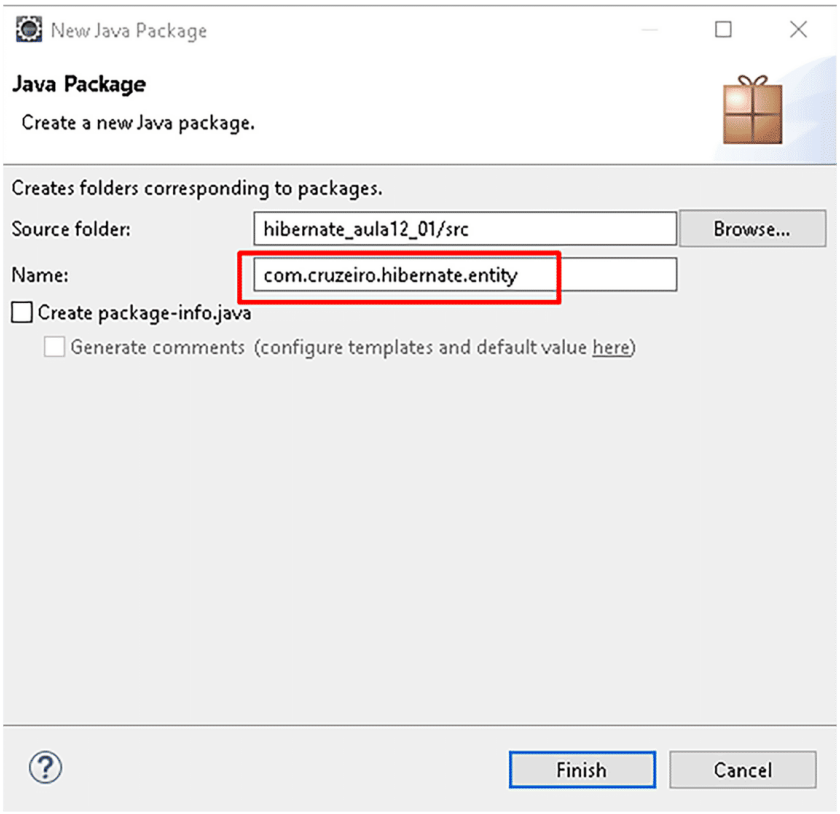


Figura 20 – Criando pacote
Fonte: Reprodução

Criar classe:

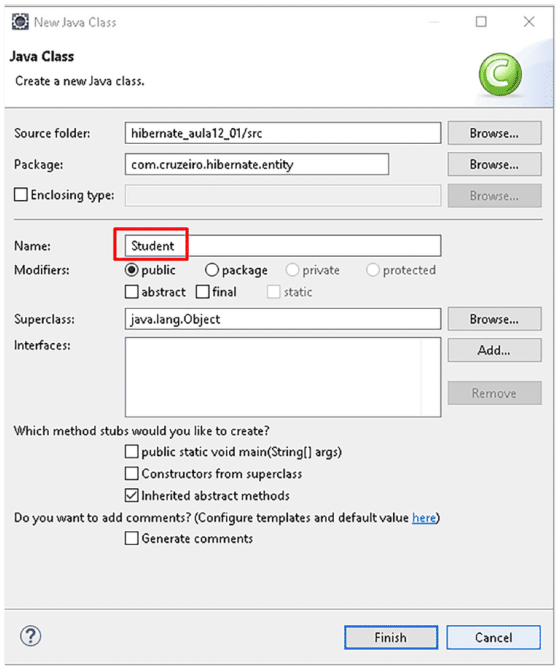


Figura 21 – Criando a classe

Fonte: Reprodução

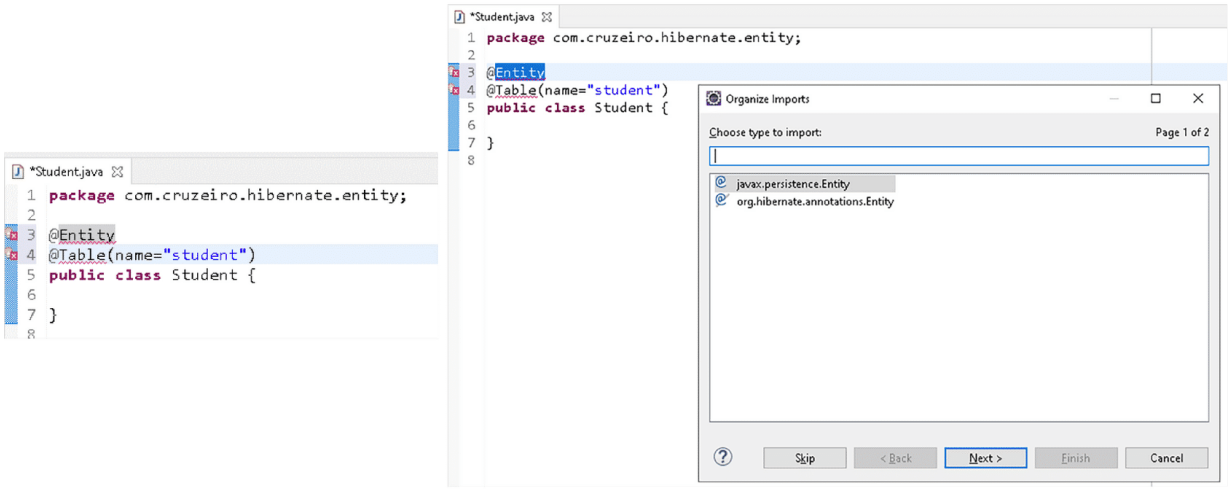


Figura 22 – Codificando a classe

Fonte: Reprodução

Dica: *CTRL* + *SHIFT* + O (*control* + *shift* + o) para realizar as importações automáticas.

Código:
Nesta Unidade, iremos abordar apenas o método de inserir. As demais funcionalidades estarão explicadas no nosso vídeo, não deixe de assistir.

Criando a classe java que representa a tabela no banco de dados relacional, observe que usamos o *@Entidy*, *@Table* e o *@Column* para mapear os campos, como explicado anteriormente. O elemento novo que vemos aqui no código é o @Id, ou seja, estamos indicando que é um uma espécie de *PK* (*Primary Key*) no banco de dados relacional.

```
package com.cruzeiro.hibernate.entity;
import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.Id;
import javax.persistence.Table;
@Entity
@Table(name="student")
public class Student {
@Id
@Column(name="id")
private int id;

@Column(name="first_name")
private String firstName;

@Column(name="last_name")
private String lastName;

@Column(name="email")
private String email;

public Student() {
super();
this.firstName = "";
this.lastName = "";
this.email = "";
```

```
}

public Student(String firstName, String lastName, String email) {
super();
this.firstName = firstName;
this.lastName = lastName;
this.email = email;
}
public int getId() {
        return id;
}
public void setId(int id) {
        this.id = id;
}
public String getFirstName() {
        return firstName;
}
public void setFirstName(String firstName) {
        this.firstName = firstName;
}
public String getLastName() {
        return lastName;
}
public void setLastName(String lastName) {
        this.lastName = lastName;
}
public String getEmail() {
        return email;
}
public void setEmail(String email) {
        this.email = email;
}
@Override
public String toString() {
        return "Student [id=" + id + ", firstName=" + firstName + ", lastNa
}
}
```

Criando o código para inserir o objeto na tabela:

Veja que, agora, para criar a conexão e saber qual banco iremos utilizar, fazemos um apontamento para o arquivo xml criado anteriormente (**hibernate.cfg.xml**). Esse arquivo possui

todas as informações necessárias para a conexão com o banco.

```
package com.cruzeiro.data;
import com.cruzeiro.hibernate.entity.Student;
public class StudentInsert {
public static void main(String[] args) {
// create session factory
SessionFactory factory = new Configuration()
.configure("hibernate.cfg.xml")
.addAnnotatedClass(Student.class)
.buildSessionFactory();
// create session
Session = factory.getCurrentSession();
try {
System.out.println("Criando objeto Students...");
Student tempStudent = new Student("Alexander", "Gobbato", "alexander@cruze
session.beginTransaction();
System.out.println("Inserindo os dados no banco...");
session.save(tempStudent);
session.getTransaction().commit();
System.out.println("Finalizado");
}
finally {
factory.close();
}
}
}
```

Vamos entender um pouco esse método de gravar estudante.

Para que serve o *Session* e o *SessionFactory*?

A principal função do *Session* é oferecer um mecanismo para autenticação dos usuários nas entidades mapeadas.

“Uma *SessionFactory* é criada por um objeto *Configuration*, na qual adquire uma informação de configuração do *Hibernate* e usa isso para gerar uma instância de *SessionFactory* apropriada. Essa informação de configuração é um arquivo externo, como veremos posteriormente.”

- HIGOR, 2013 a, n. p.

- ***session.beginTransaction*():** Essa parte do código é um segurança que estamos inserindo. A partir desse ponto, é como se estivéssemos com uma cópia dos elementos, manipulando-os. As informações ainda não estão prontas para o usuário final até que as finalizemos;

- ***session.save*():** Como estamos com os dados todos no objeto e esse objeto está mapeado para uma tabela, é como se estivéssemos colocando o dado do objeto na tabela;

- ***session.getTransaction*().*commit*():** O *getTransaction* retorna o objeto de transação atual que está aberto. Você se lembra que na explicação anterior falamos no *beginTransaction* e que estava esperando a finalização dos elementos? Então, esse comando finaliza e libera o dado para o destino final.

Para finalizarmos, veja a relação das operações disponíveis para o objeto *Session*.

Quadro 1 – Operações disponíveis para o objeto *Session*

Método	Descrição

Método	Descrição
save()	Salva um objeto na base de dados. Este método não deveria ser chamado por um objeto que foi salvo na base de dados.
saveOrUpdate()	Salva um objeto na base de dados ou atualiza a base de dados se o objeto já existe. Este método é menos eficiente que o método *save*(), pois ele necessidade fazer um *SELECT* para checar se o objeto já existe, mas não falhará se o objeto já foi salvo.
merge()	Mescla os campos de um objeto não persistente dentro de um objeto persistente apropriado (determinado pelo ID). Se tal objeto não existe na base de dados, então um é criado e salvo.
persist()	Reassocia um objeto com a sessão, dessa forma as alterações feitas no objeto serão persistidas.
get()	Recupera um objeto específico da base de dados pelo objeto identificador.
getEntityName()	Retorna o nome da entidade.
getIdentifier()	Determina o identificador para um objeto específico associado com a sessão.

Método	Descrição
load()	Carrega um objeto da base de dados pelo identificador do objeto (deveremos usar *get*() se estivermos certo que o objeto está na base de dados).
refresh()	Atualiza o estado de um objeto associado da base de dados.
update()	Atualiza a base de dados com alterações de um objeto.
delete()	Deleta um objeto da base de dados.
createFilter()	Cria um filtro (consulta) para restringir operações na base de dados.
enableFilter()	Permite um filtro nomeado nas consultas produzidas pelo *createFilter*().
disableFilter()	Disabilita um filtro nomeado.
getEnabledFilter()	Retorna um objeto filtro habilitado atualmente.
createQuery()	Cria uma consulta no Hibernate para ser aplicado na base de dados.
getNamedQuery()	Retorna uma consulta de um arquivo de mapeamento.

Método	Descrição
cancelQuery()	Cancela a execução de qualquer consulta atualmente em progresso de outra thread.
createCriteria()	Cria um objeto Criteria para operações de consulta.
beginTransaction()	Inicia uma transação.
getTransaction()	Retorna o objeto de transação atual. Este método não retorna *null* quando não há transações em progresso, o método retorna uma propriedade do objeto retornado como false.
lock()	Recebe um bloqueio da base de dados para um objeto.
contains()	Determina se um objeto específico está associado com a base de dados.
clear()	Limpa a sessão de todas as instâncias carregadas e cancela qualquer salvamento, atualização ou deleção que não tenha sido completado.
evict()	Desassocia um objeto da sessão para que as alterações posteriores a ele não sejam persistentes.

Método	Descrição
flush()	Libera todas as alterações pendentes no banco de dados - todos salvamentos, atualizações e exclusões serão realizados. Essencialmente, este método sincroniza a sessão com o banco de dados.
isOpen()	Determina se a sessão foi fechada.
isDirty()	Determina se a sessão está sincronizada com a base de dados.
getCacheMode()	Determina o modo de cache atualmente empregado.
setCacheMode()	Altera o modo de cache atualmente empregado.
getCurrentLockMode()	Determina o modo de bloqueio atualmente empregado.
setFlushMode()	Determina o modo de liberação atualmente usado. As opções são liberar depois de todos as operações, liberar quando necessário, nunca liberar, ou liberar apenas no *commit*.

Método	Descrição
setReadOnly()	Marca um objeto persistente como apenas leitura (ou como de escrita). Existe menores benefícios de performance marcando um objeto como apenas leitura, mas alterações nos estados serão ignorados até que ele seja marcado como de escrita.
close()	Fecha a sessão e a conexão com a base de dados. Libera outros recursos (como cache). Não devemos realizar qualquer operações sobre o objeto de sessão após chamar close().
getSessionFactory()	Retorna uma referência do objeto *SessionFactory* que criou a instância do Session atual.
connection()	Retorna uma referência da conexão da base de dados.
disconnect()	Desconecta da conexão atual com a base de dados.
reconnect()	Reconecta com a base de dados.
isConnected()	Determina se a conexão com a base de dados está conectada.

Fonte: Reprodução

Material Complementar

Indicações para saber mais sobre os assuntos abordados nesta Unidade:

Livros

Aprenda J2EE em 21 dias

BOND, M. *et al*. **Aprenda J2EE em 21 dias**. Tradução João Eduardo Nobrega, v. 1, p. 1488-1, 2003.

Ajax, *Rich iInternet Applications e Desenvolvimento Web* para Programadores

DEITEL, P. J.; DEITEL, H. M. **Ajax, *rich internet applications e desenvolvimento Web* para programadores**. Pearson Prentice Hall, 2009.

Core Java 1: *Fundamentals*

HORSTMANN, C. S., CORNELL, G. ***Core Java* 1: *Fundamentals***. Vol. 1. Pearson, 2015.

Lógica de Programação e Estruturas de Dados, com Aplicações em *Java*

PUGA, S; RISSETTI, G. **Lógica de programação e estruturas de dados, com aplicações em *Java*** . Pearson Educación, 2008.

Desenvolvendo *Sites* de *E-commerce*: como Criar um Eficaz e Lucrativo *Site* de *E-commerce*, Passo a Passo

SHARMA, V; SHARMA, R. **Desenvolvendo *sites* de *e-commerce*: como criar um eficaz e lucrativo *site* de *e-commerce*, passo a passo**. São Paulo: Makron, 2001.

Referências

HIGOR. ***Entendendo Hibernate Session***. ***Dev Media***, 2013a. Disponível em: <https://www.devmedia.com.br/entendendo-hibernate-session/29215#:~:text=Uma%20SessionFactory%20%C3%A9%20um%20objeto,obtido%20como%20um%20recurso%20JNDI>. Acesso em: 16/07/2022.

HIGOR. Como usar o *Hibernate Session* para manipular dados. ***Dev Media***, 2013b. Disponível em: <https://www.devmedia.com.br/como-usar-o-hibernate-session-para-manipular-dados/29248>. Acesso em: 16/07/2022.